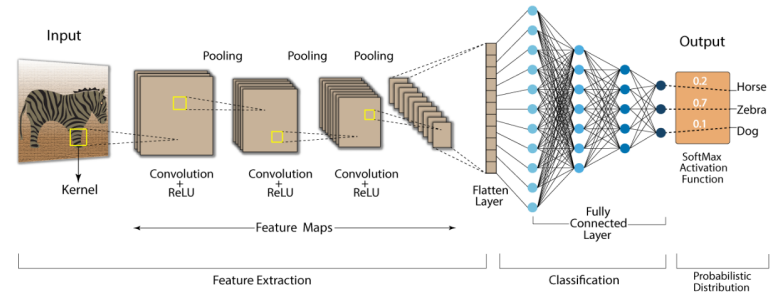


SI3003 - Inteligencia Artificial

CLASE 8 – REDES NEURONALES CONVOLUCIONALES (CNN)

Hoy

- Motivación: por qué una FC no basta para imágenes
- **Fundamentos matemáticos de la convolución**
 - Definición, multicanal, correlación cruzada
- **Padding, stride y cálculo de dimensiones**
- **ReLU y Pooling**
- **Arquitectura completa de una CNN**
- **Arquitecturas modernas (LeNet → ResNet → ViT)**
- **Regularización y buenas prácticas**
- **Transfer Learning**
- **Visualización e interpretabilidad**
- **Detección de objetos y YOLO**
- **Segmentación semántica**
- **Recursos interactivos y ejercicios**



Parte 0 — Motivación

¿Por qué no basta una red totalmente conectada?

Una imagen de $224 \times 224 \times 3$ tiene 150,528 valores de entrada. Una sola capa totalmente conectada con 1000 neuronas ya necesitaría más de 150 millones de pesos — **antes de aprender nada útil**.

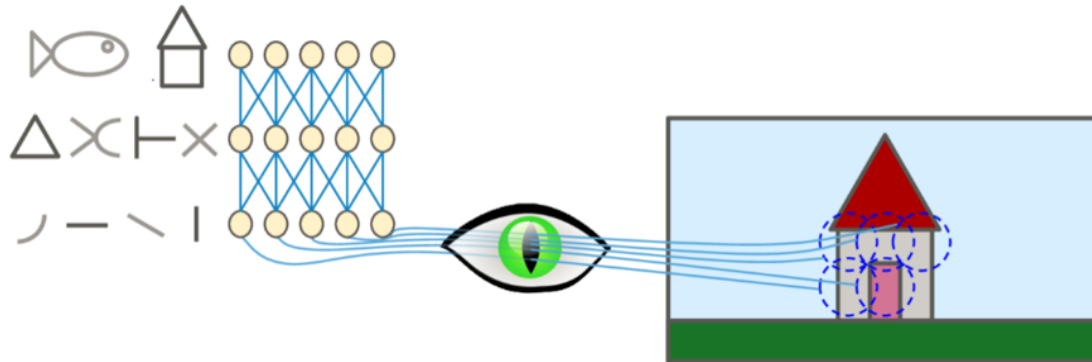
Además, una FC **ignora la estructura espacial**: si permutamos los píxeles de la misma forma en todas las imágenes, a una red FC no le importaría. Pero sabemos que un borde, una textura o un ojo son patrones *locales* que se repiten en distintas posiciones de la imagen.

La convolución explota exactamente esa estructura:

patrones locales

que se repiten, mediante dos ideas — **conectividad local** y **compartición de parámetros** (*weight sharing*).

La intuición: campos receptivos locales



Igual que la corteza visual (Hubel & Wiesel, 1959): no reconocemos un objeto mirando el ojo entero de golpe. Reconocemos

partes locales

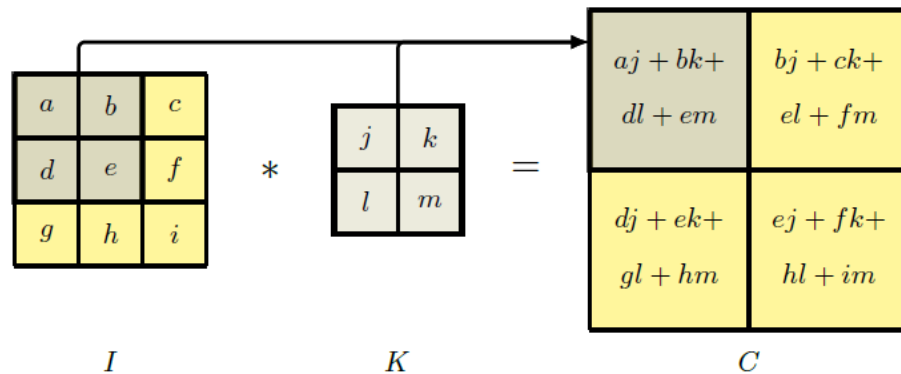
—bordes, esquinas, texturas— y las combinamos jerárquicamente en capas sucesivas hasta llegar al objeto completo.

Esa jerarquía —bordes → texturas → partes → objetos— es precisamente lo que aprenderán las capas de una CNN, sin que nadie se lo programe explícitamente.

Parte 1 — Fundamentos matemáticos de la convolución

La operación de convolución

Dada una imagen de entrada I y un kernel (filtro) K de tamaño $k \times k$, la convolución 2D discreta se define como:



$$C(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I(i + m, j + n) \cdot K(m, n)$$

Cada celda de la salida C es un **producto punto** entre el kernel y el parche de la imagen que cubre en ese momento.

Convolución multicanal (imágenes RGB)

Una imagen a color tiene C_{in} canales (para RGB, $C_{in} = 3$). El kernel también tiene esa profundidad, y la salida para el canal de salida c suma sobre todos los canales de entrada:

$$C_c(i, j) = \sum_{d=0}^{C_{in}-1} \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I_d(i+m, j+n) \cdot K_{c,d}(m, n) + b_c$$

- b_c es el **sesgo** (bias) asociado al filtro c .
- Si queremos C_{out} mapas de características distintos, necesitamos C_{out} kernels independientes, cada uno de tamaño $k \times k \times C_{in}$.

La salida de una capa convolucional no es una imagen: es un **volumen** de C_{out} mapas de características apilados.

Convolución vs. correlación cruzada

Matemáticamente, la convolución "verdadera" **invierte** el kernel antes de aplicarlo:

$$(I * K)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n)$$

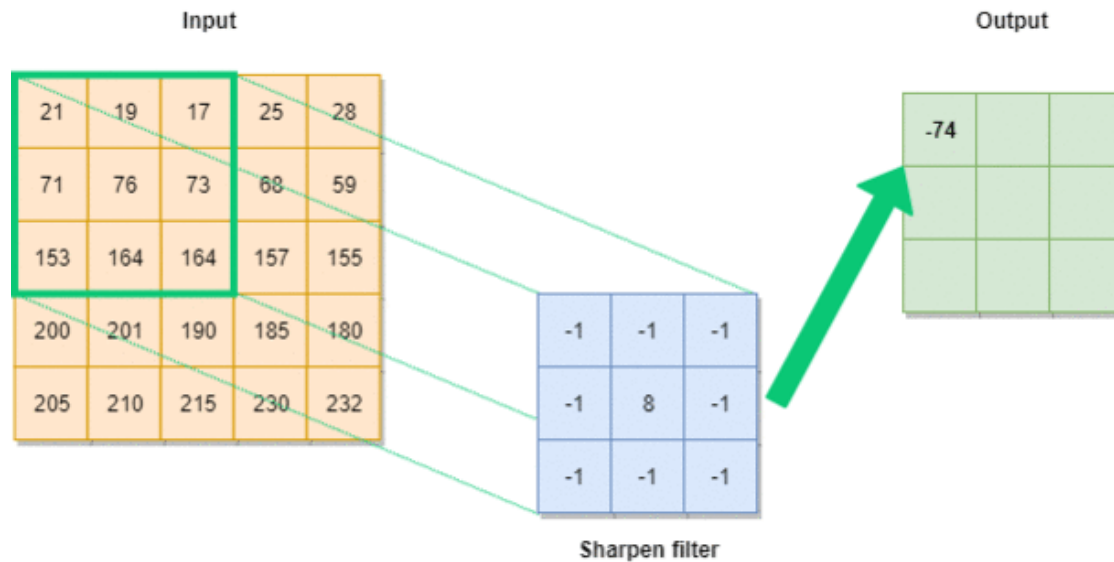
En la práctica, PyTorch y TensorFlow implementan

correlación cruzada

— sin invertir el kernel — porque para una red que *aprende* sus propios pesos da exactamente igual: si hiciera falta invertir el kernel, la red simplemente aprendería el kernel ya invertido.

Se le sigue llamando "convolución" por convención histórica, pero lo que en realidad calcula un framework de deep learning es correlación cruzada.

Ejemplo numérico: el filtro *sharpen*



AIGeekProgrammer.com © 2019

Kernel *sharpen*:

Créditos de la animación: AIGeekProgrammer.

$$K_m = \begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ 1 & 1 & 1 \end{pmatrix}$$

Filtros clásicos de detección de bordes



$$\ast \begin{pmatrix} -1 & 2 & -1 \\ -1 & 2 & -1 \\ -1 & 2 & -1 \end{pmatrix} =$$



$$\ast \begin{pmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{pmatrix} =$$



Un kernel no siempre se aprende: algunos se **diseñan a mano** para resaltar un patrón específico. Aquí el mismo kernel, transpuesto, resalta bordes verticales a la izquierda y horizontales a la derecha.

Este es el puente conceptual clave: una CNN **aprende** kernels como estos —y muchos más complejos— en lugar de que un humano los diseñe a mano.

Parte 2 — Padding, Stride y Dimensiones

Padding: por qué agregar ceros alrededor

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

114				

Sin padding, dos problemas:

- La imagen se **encoge** en cada capa convolucional.
- Los píxeles de las **esquinas y bordes** participan en muchas menos convoluciones que los del centro

Stride: el tamaño del salto

El **stride** S es cuánto se desplaza el kernel en cada paso.

- $S = 1$: el kernel se desliza *píxel por píxel* — máxima resolución de salida.
- $S = 2$: el kernel *salta uno* — la salida se reduce a la mitad aproximadamente, sin necesidad de una capa de pooling aparte.

El stride es una forma alternativa de hacer *downsampling*: en arquitecturas modernas (ResNet) es común usar convoluciones con stride 2 en vez de max-pooling para reducir la resolución.

Fórmula general del tamaño de salida

Para una entrada $W \times W$, kernel $K \times K$, padding P y stride S :

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

Se aplica igual a alto y ancho (por separado, si la entrada no es cuadrada).

Ejemplo 1 — entrada 7×7 , $K = 3$, $P = 0$, $S = 1$:

$$O = \frac{7 - 3 + 0}{1} + 1 = 5$$

Salida: 5×5 .

Ejemplo 2 — *same padding*, $K = 3$, $P = 1$, $S = 1$:

$$O = \frac{7 - 3 + 2}{1} + 1 = 7$$

Salida: 7×7 (igual a la entrada).

Número de parámetros de una capa convolucional

Para una capa con C_{in} canales de entrada y C_{out} filtros de tamaño $K \times K$:

$$\text{Parámetros} = (K \times K \times C_{in} + 1) \times C_{out}$$

El $+1$ es el bias de cada filtro.

Ejemplo: capa con entrada de 64 canales, 128 filtros de 3×3 :

$$(3 \times 3 \times 64 + 1) \times 128 = (576 + 1) \times 128 = 73,856 \text{ parámetros}$$

Comparen con una capa FC equivalente sobre la misma entrada aplanada: sería órdenes de magnitud más grande. Esa diferencia es **weight sharing** en acción.

Parte 3 — ReLU y Pooling

La función de activación ReLU

$$\text{ReLU}(x) = \max(0, x)$$

Derivada (la que usa backpropagation):

$$\text{ReLU}'(x) = \begin{cases} 1 & \text{si } x > 0 \\ 0 & \text{si } x \leq 0 \end{cases}$$

Ventajas frente a sigmoide/tanh (vistas en la Clase 7):

- No satura del lado positivo → mitiga el *vanishing gradient*.
- Cómputo trivial: un solo max.
- Induce *sparsity*: muchas activaciones quedan exactamente en cero.

Variantes a conocer: **Leaky ReLU**, $\text{LReLU}(x) = \max(\alpha x, x)$ con α pequeño (p. ej. 0.01) — evita neuronas "muertas" que siempre dan cero. **GELU**, usada en arquitecturas modernas tipo Transformer.

Pooling: max y average

Reduce la resolución espacial del mapa de características, manteniendo la información más relevante.

Max pooling:

$$P(i, j) = \max_{(m, n) \in \text{región}} I(i + m, j + n)$$

Conserva la activación *más fuerte* de cada región
— típicamente la más usada en la práctica.

Average pooling:

$$P(i, j) = \frac{1}{k^2} \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I(i + m, j + n)$$

Conserva el *promedio* — más suave, útil en la última capa (ver GAP).

El tamaño de salida sigue la **misma fórmula** de la convolución (el pooling no tiene parámetros aprendibles):

$$O = \left\lfloor \frac{W - K}{S} \right\rfloor + 1$$

Global Average Pooling (GAP)

En vez de aplanar (*flatten*) el último volumen de mapas de características y conectarlo a una capa FC enorme, GAP colapsa

cada mapa completo

a un solo número —su promedio—.

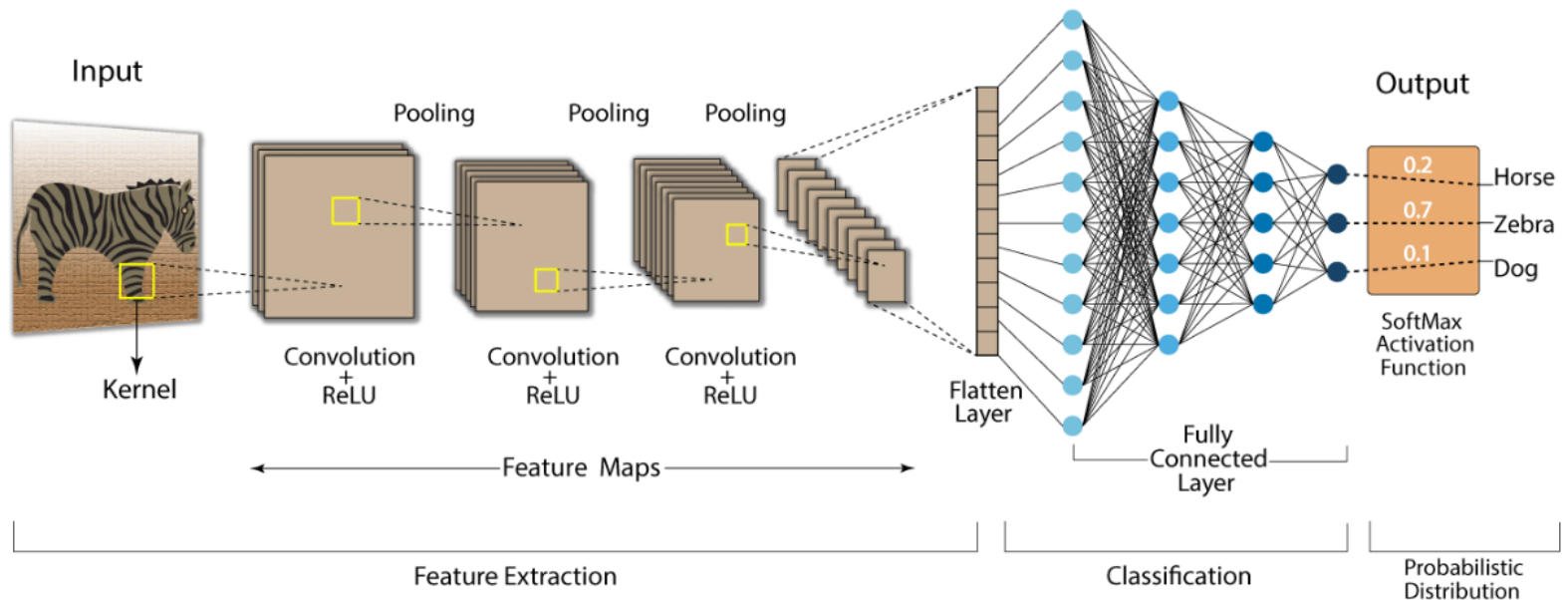
$$\text{GAP}(F_c) = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W F_c(i, j)$$

Si el último volumen tiene C_{out} canales, GAP produce directamente un vector de C_{out} valores — listo para una capa softmax pequeña.

Ventaja: elimina la capa FC más costosa de la red (recordar que en VGG-16 la primera FC concentraba **103 millones** de los 138 millones de parámetros totales). GAP es estándar en ResNet, Inception y arquitecturas posteriores.

Parte 4 — Arquitectura completa de una CNN

El flujo completo, de la imagen a la predicción

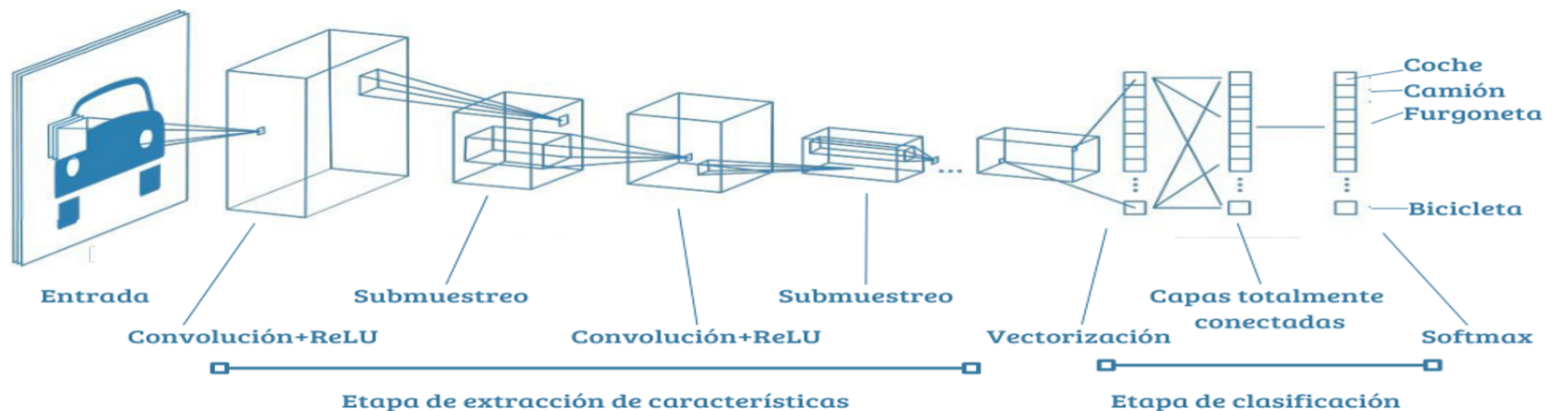


Input $\rightarrow$ [Conv $\rightarrow$ ReLU $\rightarrow$ Pool] $\times n \rightarrow$ Flatten (o GAP) $\rightarrow$ FC $\rightarrow$ Softmax

Dos etapas bien diferenciadas:

- **Extracción de características:** las capas convolucionales (aprenden *qué mirar*).

El mismo flujo, con vocabulario en español



- *Submuestreo* = pooling.
- *Vectorización* = flatten.
- *Capas totalmente conectadas* = fully connected (FC).

La salida sigue siendo **softmax**, la misma capa de la Clase 7 — nada nuevo ahí. Lo único nuevo hoy es

cómo se construyen las características

Softmax y la función de pérdida (repaso)

Igual que en la Clase 7, la capa final produce probabilidades:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

Y se entrena minimizando la **entropía cruzada**:

$$\mathcal{L} = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

donde y_i es la etiqueta verdadera (codificación *one-hot*) y $\hat{y}_i$ la probabilidad predicha para la clase i .

Nada de esto cambió respecto a la Clase 7. Lo que cambió es **quién produce** los valores z_i que entran al softmax: ya no una sola capa FC sobre features manuales, sino toda la pila de capas convolucionales.

Ejercicio guiado: calculando dimensiones capa por capa

Aplicando la fórmula $O = \lfloor (W - K + 2P)/S \rfloor + 1$ a una red tipo VGG simplificada:

Capa	Entrada	Kernel	Stride Padding		Salida
Conv1	$224 \times 224 \times 3$	3×3 , 64 filtros	1	same	$224 \times 224 \times 64$
Pool1	$224 \times 224 \times 64$	2×2	2	0	$112 \times 112 \times 64$
Conv2	$112 \times 112 \times 64$	3×3 , 128 filtros	1	same	$112 \times 112 \times 128$
Pool2	$112 \times 112 \times 128$	2×2	2	0	$56 \times 56 \times 128$

Ejercicio en clase: continuar la tabla hasta llegar a la capa FC, verificando cada fila con la fórmula, y calcular el número total de parámetros de la red completa (ver Bloque C de ejercicios).

Parte 5 — Arquitecturas modernas

De LeNet a los Transformers: línea de tiempo

Arquitectura	Año	Aporte clave
LeNet-5	1998	Primera CNN práctica (dígitos manuscritos)
AlexNet	2012	ReLU, dropout, entrenamiento en GPU, ImageNet
VGG-16/19	2014	Apilar filtros pequeños (3×3) en profundidad
GoogLeNet (Inception)	2014	Módulos Inception: convoluciones en paralelo de distinto tamaño
ResNet	2015	Conexiones residuales (<i>skip connections</i>)
EfficientNet	2019	Escalado compuesto (profundidad / ancho / resolución)
Vision Transformer (ViT)	2020	Reemplaza la convolución por atención

VGG-16 (la que vimos en el capítulo base) es de **2014**. Todo lo que viene después de esa fila resuelve una limitación concreta de apilar capas simplemente más y más profundo.

ResNet: el bloque residual

El problema con redes muy profundas: apilar más capas normales **empeora** el error de entrenamiento (no es sobreajuste — es que el gradiente se degrada en el camino).

La solución de ResNet: en vez de aprender $F(x)$ directamente, la capa aprende el **residuo** y se suma la entrada original sin modificar:

$$y = F(x, W_i) + x$$

donde F es el mapeo aprendido (conv $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ conv $\rightarrow$ BN) y x se suma mediante un *atajo de identidad* (*identity shortcut*).

Si la capa no tiene nada útil que aprender, basta con que $F(x, W_i) \rightarrow 0$ y la salida es simplemente x : la conexión residual nunca puede ser **peor** que no tener esa capa. Esto permite entrenar redes de cientos de capas sin degradación.

Batch Normalization

Estabiliza y acelera el entrenamiento normalizando las activaciones **dentro de cada mini-batch**:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y^{(k)} = \gamma \hat{x}^{(k)} + \beta$$

- μ_B, σ_B^2 : media y varianza del mini-batch actual.
- γ, β : parámetros **aprendibles** que permiten a la red deshacer la normalización si le conviene.
- ϵ : constante pequeña para evitar dividir por cero.

Batch Norm es hoy casi omnipresente en CNNs modernas (incluida ResNet): reduce la sensibilidad a la tasa de aprendizaje inicial y actúa como un regularizador leve.

Parte 6 — Regularización y buenas prácticas

Cómo evitar el sobreajuste en una CNN

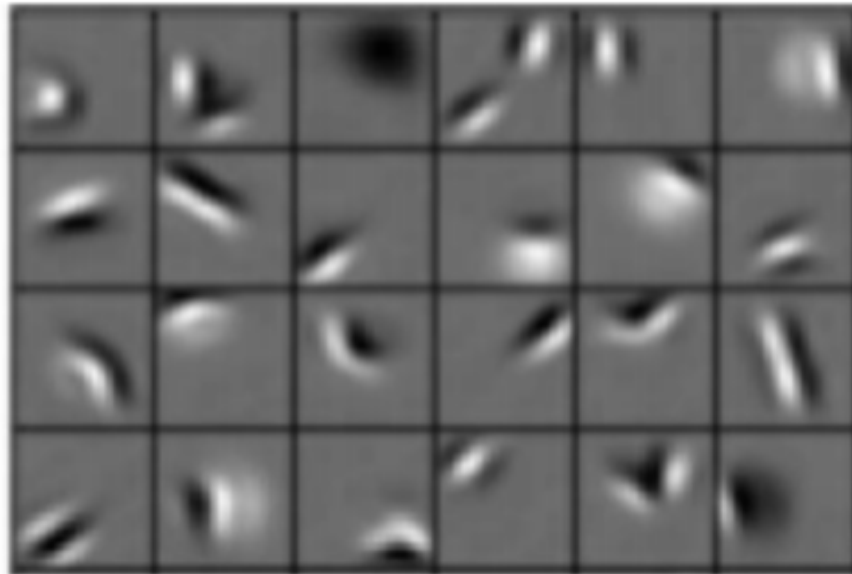
- **Data augmentation:** rotar, voltear, recortar, cambiar brillo/contraste de las imágenes de entrenamiento — aumenta la variedad efectiva del dataset sin recolectar más datos.
- **Dropout:** apaga aleatoriamente una fracción p de neuronas durante el entrenamiento (típico en las capas FC finales).
- **Batch Normalization:** además de acelerar el entrenamiento (Parte 5), actúa como regularizador leve.
- **Weight decay (L2):** penaliza pesos grandes agregando $\lambda \|\mathbf{w}\|^2$ a la función de costo.
- **Early stopping:** detener el entrenamiento cuando el error de *validación* deja de mejorar, aunque el de entrenamiento siga bajando.

Ninguna de estas técnicas cambia el modelo f ni la función de costo $\mathcal{L}$ de fondo — todas actúan *alrededor* del entrenamiento para que el mínimo que se encuentre generalice mejor.

Parte 7 — Transfer Learning

Por qué reutilizar una red ya entrenada

Las primeras capas de una CNN entrenada en un dataset grande (ImageNet, 1.3 millones de imágenes, 1000 clases) aprenden filtros **genéricos**: bordes, texturas, colores, formas simples.



Esos filtros son **reutilizables** en casi cualquier tarea visual, aunque el dataset final sea completamente distinto al original (rayos X, defectos industriales, especies de plantas, etc.).

Dos estrategias de transfer learning

Estrategia	Qué se congela	Cuándo usarla
Feature extraction	Toda la red base (solo se entrena el clasificador nuevo)	Dataset propio pequeño, similar al original
Fine-tuning parcial	Solo las primeras capas; se reentrenan las últimas con <i>learning rate</i> bajo	Dataset propio mediano, algo distinto al original
Fine-tuning completo	Nada — se reentrena todo con LR muy bajo	Dataset grande, muy distinto al dominio original

Regla práctica: mientras **más pequeño** sea tu dataset y **más parecido** al dominio original (fotos naturales), más capas conviene *congelar*.

Ejemplo práctico propuesto

1. Tomar una red preentrenada en ImageNet (ResNet-18 o VGG-16).
2. Reemplazar la última capa (la que tenía 1000 salidas) por una nueva capa softmax con C clases del dataset propio.
3. Entrenar primero solo esa capa nueva (*feature extraction*).
4. Comparar el *accuracy* contra descongelar las últimas dos o tres capas convolucionales y reentrenarlas con LR bajo (*fine-tuning*).
5. Comparar ambos resultados contra entrenar la

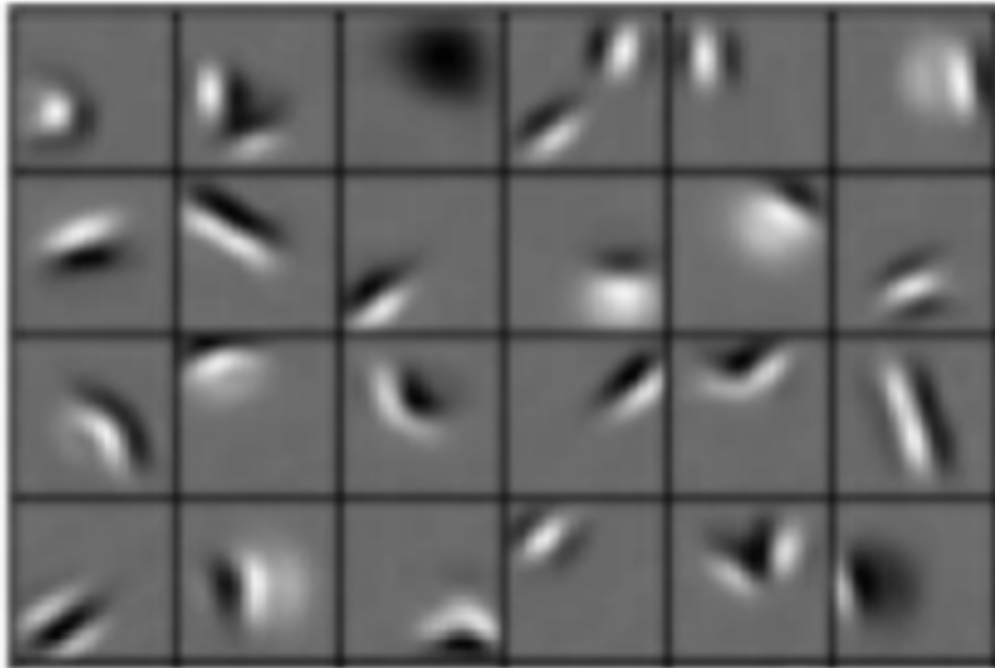
misma arquitectura desde cero

con el mismo dataset pequeño.

El resultado esperado: entrenar desde cero con pocos datos da el peor desempeño — es la evidencia más directa de por qué transfer learning es la práctica estándar en visión por computador.

Parte 8 — Visualización e interpretabilidad

Qué aprende la primera capa



Los filtros de la primera capa convolucional son directamente visualizables como pequeñas imágenes — y se parecen sorprendentemente a los filtros de Gabor, usados desde hace décadas en procesamiento de imágenes para detectar bordes orientados.

Esto **no** implica que la CNN imite la corteza visual: filtros similares emergen de casi cualquier método estadístico aplicado a imágenes naturales, porque reflejan las estadísticas de las imágenes mismas, no un

La jerarquía de capas intermedias



Capa intermedia: partes (ojos, texturas)



Capa profunda: objetos casi completos (rostros)

Esta es la jerarquía que anticipamos en la Parte 0: bordes (capa 1) → texturas y partes (capas intermedias) → objetos casi completos (capas profundas). Nadie programó esta progresión: emergió del entrenamiento.

Saliency maps y Grad-CAM

Otra pregunta: dada una imagen concreta, **qué región** fue decisiva para la predicción de la red.

El método **Grad-CAM** calcula, para la clase predicha c , el promedio de las derivadas de la salida $a^{(c)}$ (antes del softmax) respecto a las activaciones $a_{ij}^{(k)}$ del último mapa convolucional k :

$$\alpha_k = \frac{1}{M_k} \sum_i \sum_j \frac{\partial a^{(c)}}{\partial a_{ij}^{(k)}}$$

Esos promedios ponderan la suma de todos los canales, dando un mapa de calor del mismo tamaño que la última capa convolucional:

$$\mathbf{L} = \sum_k \alpha_k \mathbf{A}^{(k)}$$

Ese mapa $\mathbf{L}$, superpuesto sobre la imagen original, muestra exactamente qué región "miró" la red para decidir su predicción — muy útil para depurar y para explicar decisiones a usuarios no técnicos.

Ataques adversariales

Los mismos gradientes que entrenan la red pueden usarse para **engañarla**: modificaciones imperceptibles al ojo humano que causan una mala clasificación con alta confianza (*fast gradient sign method*):

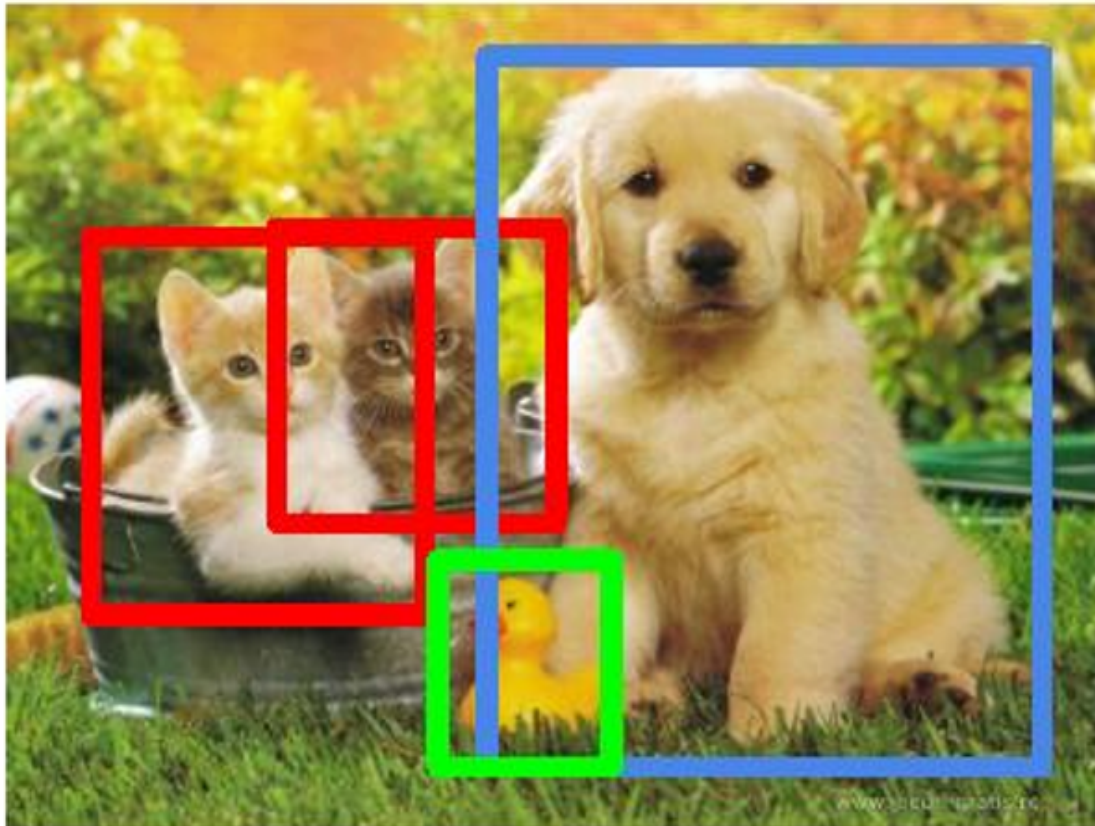
$$\mathbf{x}' = \mathbf{x} + \epsilon \operatorname{sign}(\nabla_{\mathbf{x}} E(\mathbf{x}, t))$$

donde t es la etiqueta verdadera, E el error (p. ej. entropía cruzada) y ϵ un paso pequeño que mantiene el cambio imperceptible.

El caso clásico: una foto de panda clasificada correctamente con 57.7% de confianza, tras sumarle un ruido de magnitud $\epsilon = 0.007$ (también imperceptible), se clasifica como gibón con 99.3% de confianza. Existen incluso versiones físicas: señales de alto modificadas para ser leídas como límites de velocidad por una CNN.

Parte 9 — Detección de objetos y YOLO

De clasificar a localizar: bounding boxes



Intersection-over-Union (IoU)

Para medir qué tan bueno es un bounding box predicho respecto al *ground truth* (etiquetado por un humano):

$$\text{IoU} = \frac{\text{Área}(B_{pred} \cap B_{gt})}{\text{Área}(B_{pred} \cup B_{gt})}$$

- $\text{IoU} = 1$: coincidencia perfecta.
- $\text{IoU} = 0$: los rectángulos no se superponen.
- En la práctica, se suele considerar una detección **correcta** si $\text{IoU} > 0.5$.

IoU penaliza tanto un box *demasiado grande* (que cubre área de más) como uno *demasiado pequeño* (que no cubre todo el objeto) — a diferencia de solo medir el área de superposición.

Non-max suppression

Al escanear una imagen, es común obtener **múltiples detecciones** del mismo objeto en posiciones cercanas. El algoritmo:

1. Eliminar todos los boxes cuya probabilidad esté por debajo de un umbral (p. ej. 0.7).
2. Tomar el box de **mayor probabilidad** como detección exitosa.
3. Eliminar cualquier otro box cuyo IoU con ese box exceda un umbral (p. ej. 0.5) — son la misma detección repetida.
4. Repetir con los boxes restantes hasta que todos hayan sido declarados exitosos o descartados.

Esto se aplica **por cada clase por separado**: dos detecciones superpuestas de clases distintas (p. ej. "perro" y "pelota") *no* se eliminan entre sí.

YOLO: You Only Look Once

A diferencia de *sliding window* o R-CNN (que evalúan la red muchas veces sobre distintas regiones), YOLO trata la detección como

un único problema de regresión

:

- Divide la imagen en una grilla $S \times S$.
- Cada celda de la grilla predice directamente:

Salida por celda = $(b_x, b_y, b_W, b_H, \text{confianza}, p_1, \dots, p_C)$

- Una **única pasada** por la red produce todas las detecciones de la imagen — de ahí el nombre.

Esto lo hace mucho más rápido que los enfoques de *sliding window* + CNN completa en cada ventana, al costo de ser algo menos preciso en objetos muy pequeños o muy juntos entre sí.

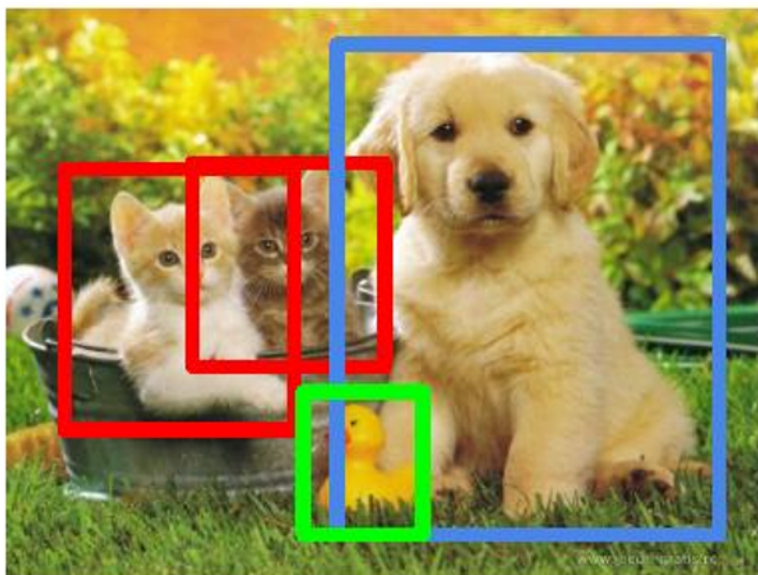
La función de pérdida de YOLO (simplificada)

$$\mathcal{L} = \lambda_{coord} \sum_i 1_i^{obj} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] + \lambda_{coord} \sum_i 1_i^{obj} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right]$$

- Primer término: error de **posición** del centro del box.
- Segundo término: error de **tamaño** — nótese la raíz cuadrada, que penaliza proporcionalmente menos los errores en boxes grandes que en boxes pequeños.
- 1_i^{obj} vale 1 solo si hay un objeto en esa celda —los términos de coordenadas **no** se penalizan en celdas vacías.
- Términos adicionales (omitidos aquí) penalizan el error de confianza y el de clasificación, de forma análoga a la entropía cruzada de la Clase 7.

Parte 10 — Segmentación semántica

De cajas a contornos: segmentación por instancia



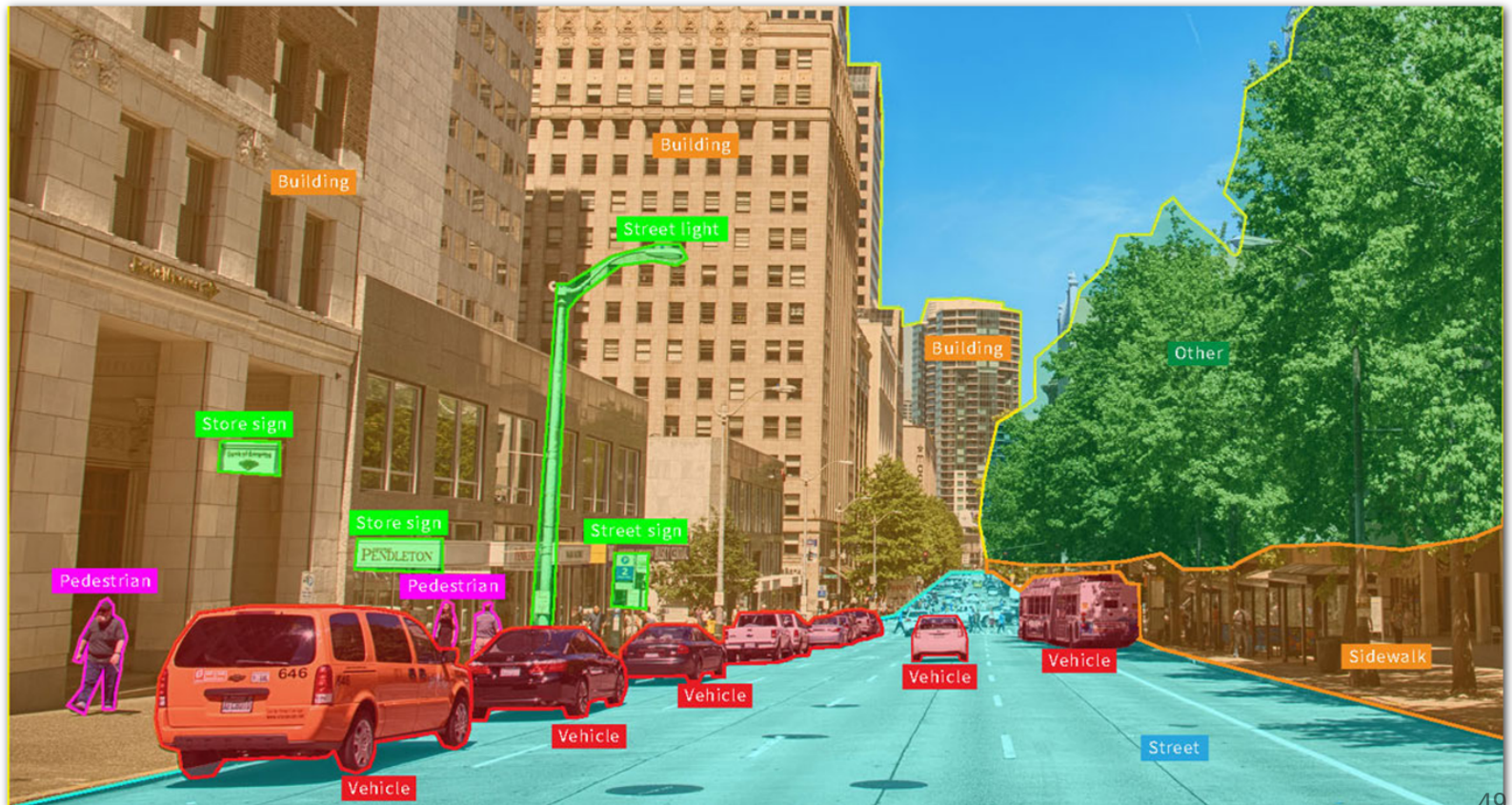
Detección: bounding boxes



Segmentación: contorno exacto de cada instancia

La segmentación va más allá del rectángulo: asigna una etiqueta a **cada píxel**, distinguiendo la silueta exacta del objeto del fondo —y, en segmentación por *instancia*, distinguiendo también un perro de otro

Segmentación semántica: todo pixel tiene una clase



Recursos interactivos

Enlaces para explorar por su cuenta

Recurso

[4.1 Convolutions — rramosp](#)

[CNN Explainer](#)

[Image Kernels Explained Visually](#)

[Zero-Padding in CNNs — Medium](#)

[CIFAR Confusion Matrix — ml4a](#)

[AI Geek Programmer — CNN arquitectura parte 2](#)

[Analytics Vidhya — Basics of CNN](#)

Uso sugerido

Repaso matemático interactivo de la convolución

Visualización interactiva de una CNN completa, capa por capa

Probar distintos kernels (blur, sharpen, edge) en tiempo real

Repaso conceptual de padding

Explorar errores reales de clasificación de una CNN

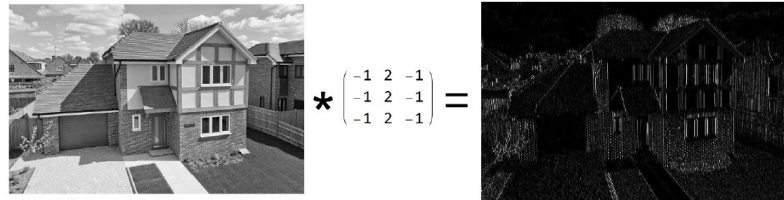
Repaso de arquitectura con ejemplos numéricos

Repaso introductorio accesible

Recomendación de uso en clase: proyectar [CNN Explainer](#) en vivo y subir una foto propia — es la forma más directa de ver todo lo visto hoy funcionando junto.

Ejercicios propuestos

Bloque A — Convolución manual con distintos kernels



1. Dada una matriz 5×5 (provista en clase), aplicar manualmente:

- Identidad: $\begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$

- *Sharpen*: $\begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}$

- *Blur* (promedio): $\frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$

- Sobel vertical y horizontal.

- Calcular a mano al menos 2 celdas de salida: verificar el resto con

Bloque B — Padding y stride

0	0	0	0	0	0	0	0
0	60	113	56	139	85	0	0
0	73	121	54	84	128	0	0
0	131	99	70	129	127	0	0
0	80	57	115	69	134	0	0
0	104	126	123	95	130	0	0
0	0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

114				

1. Para una imagen 10×10 , calcular el tamaño de salida con:

- $K = 3, S = 1, P = 0$
- $K = 3, S = 2, P = 1$
- $K = 5, S = 1, P = 2$ (*same padding*)

Verificar cada resultado con $O = \lfloor (W - K + 2P)/S \rfloor + 1$ y con código.

2. Implementar en NumPy una función `conv2d(image, kernel, stride, padding)` desde cero (sin librerías de deep learning) y validarla contra `scipy.signal.convolve2d`.

Bloque C — Arquitectura y dimensiones

1. Completar la tabla de dimensiones de la Parte 4 (Ejercicio guiado) hasta la capa de salida, y calcular el

número total de parámetros

de la red completa usando

$$\text{Parámetros} = (K \times K \times C_{in} + 1) \times C_{out}$$

para cada capa convolucional.

Bloque D — Visión práctica con CNN entrenada



1. Cargar una CNN preentrenada (ResNet-18 de `torchvision`) y clasificar 5 imágenes propias. Visualizar los mapas de activación de la primera capa convolucional (comparar con la Figura 6 de esta clase).
2. Explorar [CNN Explainer](#), subiendo una imagen propia y observando cómo cambian las activaciones capa por capa.

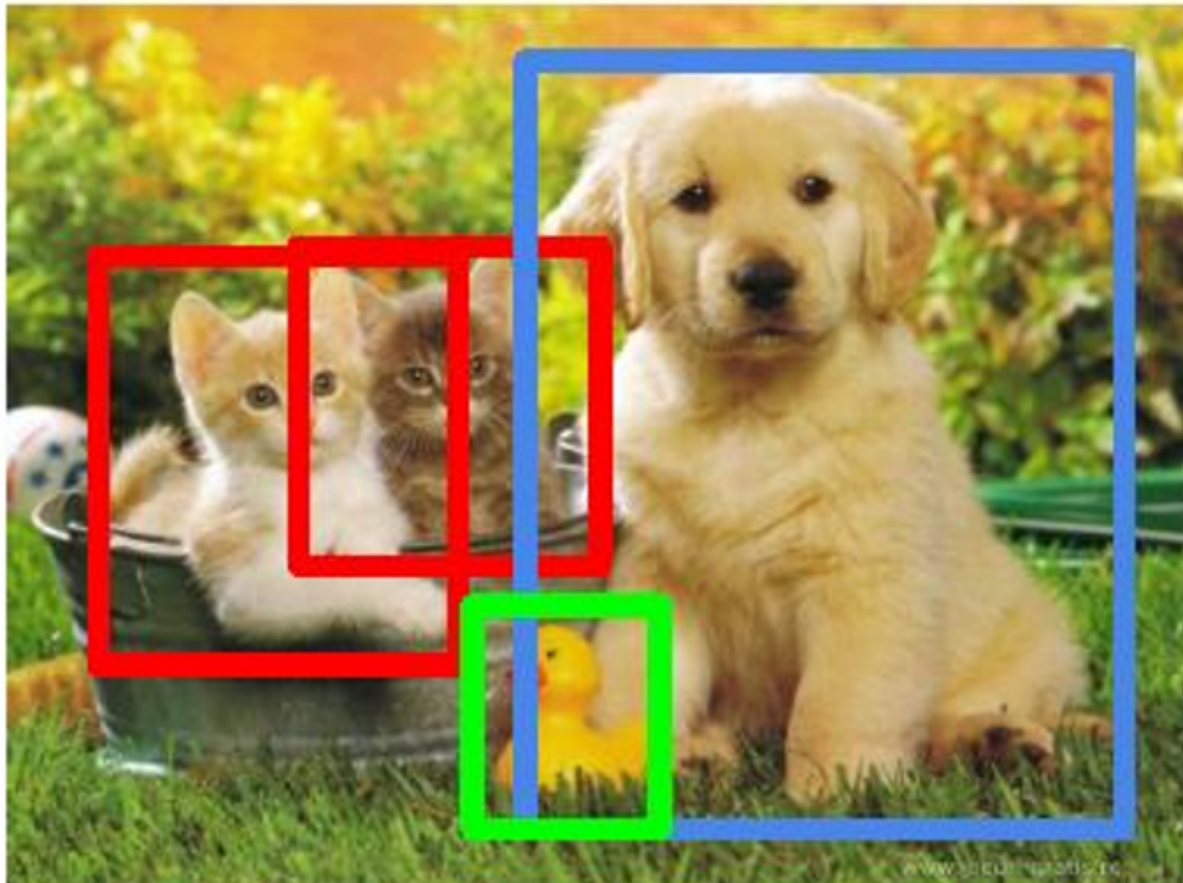
Bloque E — Transfer learning

1. Tomar un dataset pequeño (p. ej. 200 imágenes, 2 clases) y entrenar un clasificador usando **feature extraction** sobre una red preentrenada.
2. Comparar el *accuracy* contra entrenar la

misma red desde cero

con el mismo dataset pequeño. Reportar la diferencia y explicarla con lo visto en la Parte 7.

Bloque F — YOLO y detección de objetos



Bloque G — Análisis de errores

1. Usando [CIFAR Confusion Matrix](#), identificar qué clases se confunden más entre sí y proponer una hipótesis de por qué (basada en similitud visual de bajo nivel: bordes, texturas, color — conectar con la Parte 8).

Resumen

- Una CNN reemplaza la conectividad total de una FC por

conectividad local

y **compartición de parámetros** — la misma neurona (kernel) se aplica en toda la imagen.

- La convolución produce, para cada posición, un **producto punto** entre el kernel y el parche de la imagen: $C(i, j) = \sum_m \sum_n I(i + m, j + n)K(m, n)$.
- **Padding** evita que la imagen se encoja y que los bordes se subrepresenten; **stride** controla el tamaño del salto. El tamaño de salida siempre es $O = \lfloor (W - K + 2P)/S \rfloor + 1$.
- **ReLU** sigue siendo la activación por defecto; **pooling** (max o average) reduce la resolución espacial y aporta invariancia a pequeñas traslaciones.
- La arquitectura completa repite $[\text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool}]$ varias veces y termina igual que la Clase 7: FC + softmax + entropía cruzada.
- **ResNet** resolvió la degradación de redes muy profundas con conexiones residuales $y = F(x) + x$; **Batch Norm** estabiliza el entrenamiento.
- **Transfer learning** reutiliza filtros genéricos aprendidos en ImageNet — la práctica estándar cuando el dataset propio es pequeño.
- Grad-CAM y los ataques adversariales muestran que se puede —y a veces se debe— mirar *dentro* de la red, no solo confiar en su *accuracy*.
- **YOLO** convierte la detección en un problema de regresión de una sola pasada; la **segmentación**

Fin de la Clase 8

Próxima clase: Redes Neuronales Recurrentes (RNN) y Modelos de Secuencia